KoBLEX / ParSeR — Reproduction Report

Nov 5, 2026 · Arihant Rakhecha (2023B1A70998H)

Summary

The paper's headline claim reproduces: ParSeR's retrieval F1 with GPT-4o is 59.4 against 21.5 for standard retrieval, a gap of 37.9, matching the reported +37.91. In the saved outputs it matches the **retrieval** F1 gap; the answer F1 gap is smaller (46.1 vs 26.8).

What was reproduced, and how:

- **Scored all 46 saved result files** with the repo's own metric functions (the official eval script crashes on them; see Issues).
- **Re-ran the standard-retrieval BM25 baseline from scratch** on CPU: F1 21.4 vs 21.5 saved, with identical retrieved provisions for 218 of 226 questions.
- **Prepared a fixed Colab notebook** for the full ParSeR run (GPU reranker + GPT-4o), which needs an API key; results to be added once it runs.

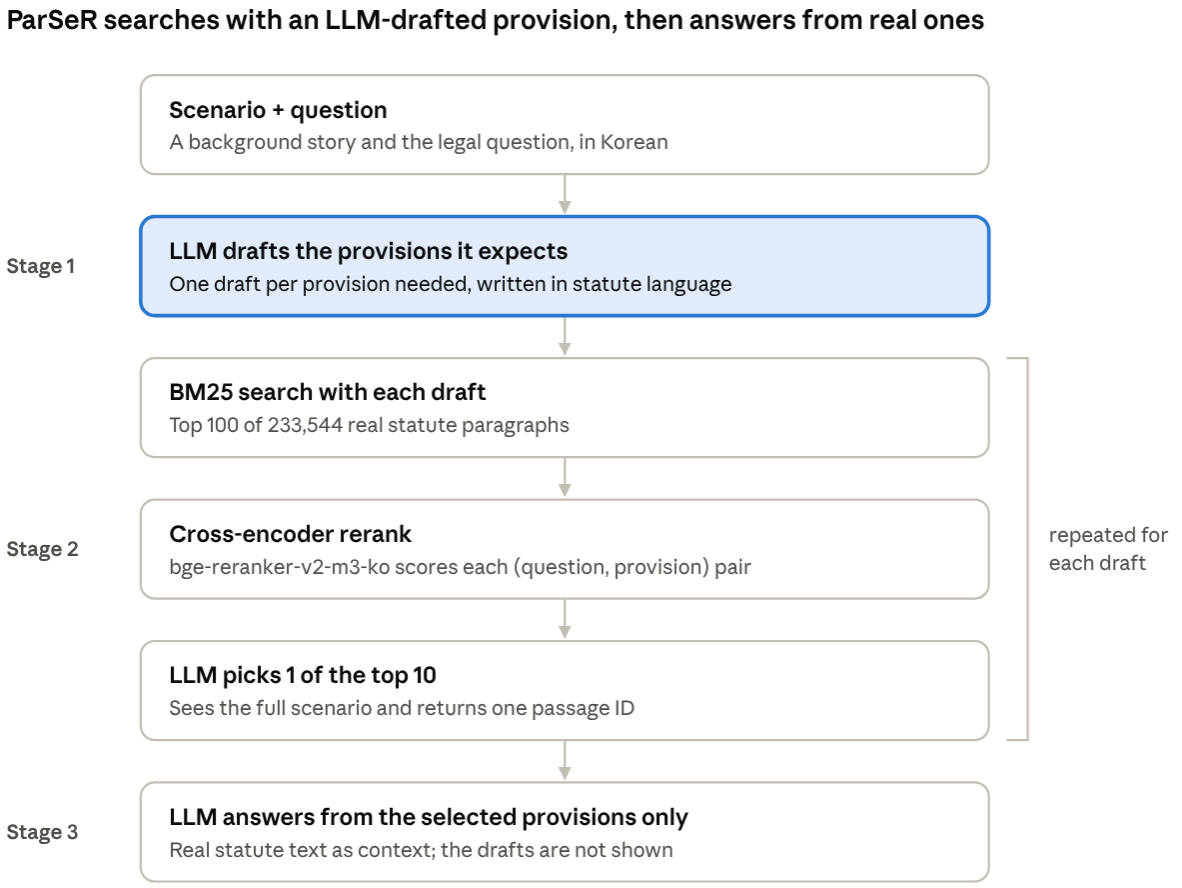
Four findings worth discussing:

1. **Wrong retrieval is worse than no retrieval.** GPT-4o with BM25 context scores 26.8 answer F1, below 36.2 with no context at all; 26 of 226 answers refuse because the context is irrelevant.
2. **A bug in the cleaned-up ParSeR code.** `parser/openai/selection_retrieval.py` (and the vLLM version) picks every selected provision from the *first* sub-query's candidates. The original script does it correctly.
3. **BM25 is handicapped by English tokenization of Korean.** Switching to Korean character bigrams, with no LLM, raises baseline retrieval F1 from 21.4 to 37.4 and recall@100 from 51.8 to 75.6.
4. **ParSeR's typical failure is a near miss**: the right statute, the wrong neighbouring provision (e.g. a 2-year limitation period instead of 1 year). It retrieved none of the gold provisions for 36 of 226 questions.

The benchmark and the method

KoBLEX tests whether a system can answer open-ended Korean legal questions and cite the exact statute paragraphs that support the answer. It has 226 scenario-based questions, each needing 1 to 3 provisions, built by LLM generation and revised by legal experts.

The paper's method, ParSeR, rests on one idea: a question and a statute use different words, so the LLM first writes the provision it expects, and that draft is used as the search query.



ParSeR pipeline, from the paper and parser/openai/ code · 3 stages

The new parts are the draft in stage 1 and the three-step search it feeds (BM25, rerank, LLM selection), run once per draft; stage 3 is ordinary grounded answering. The baselines it is compared with are standard prompting and chain-of-thought (with and without one BM25 retrieval), Self-Ask, IRCoT, FLARE, ProbTree and BeamAggr, each on GPT-4o, Qwen and Exaone.

Setup

Everything below uses commit `1a2d225` of daehuikim/KoBLEX (3 Nov 2025) and the paper KoBLEX: Open Legal Question Answering with Multi-hop Reasoning, EMNLP 2025.

Part	Where it ran	Status
Scoring saved outputs (token F1, retrieval P/R/F1/EM)	CPU, Python 3.13	Done

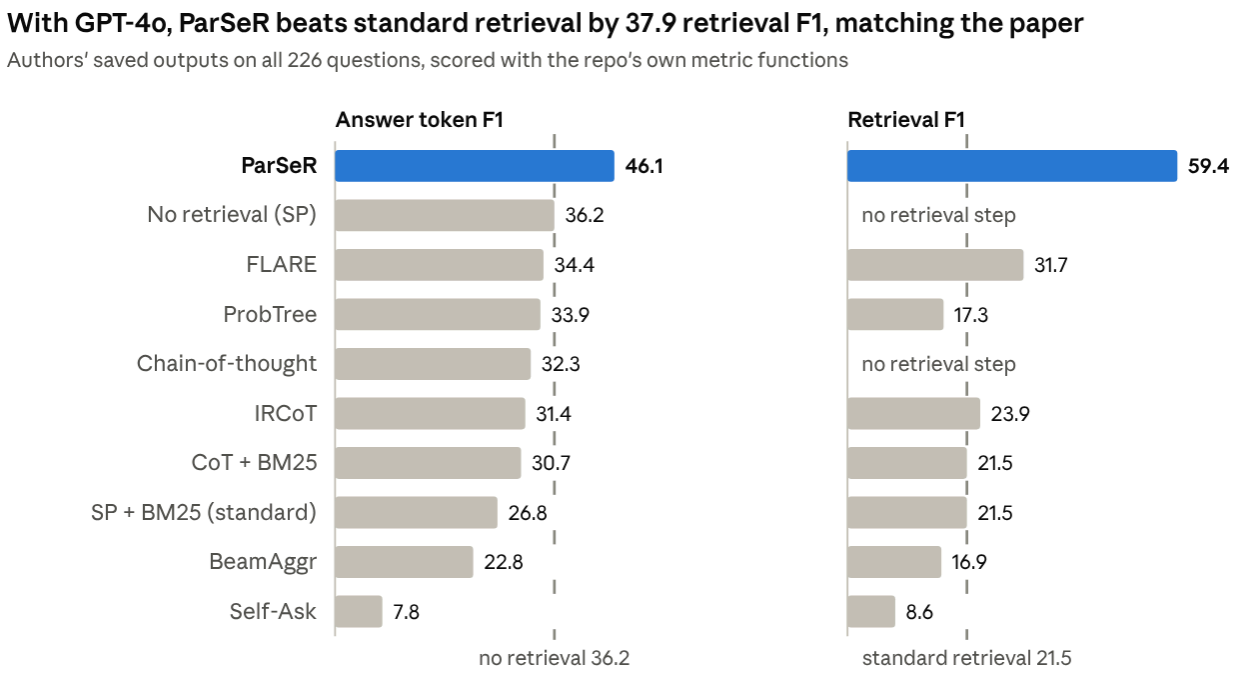
Part	Where it ran	Status
BM25 standard-retrieval baseline, re-run	CPU, `bm25s` 0.3.16, 2 cores, 7 GB RAM	Done, 12 s to index 233,544 provisions
Korean bigram BM25 (own experiment)	CPU	Done
ParSeR end to end (GPT-4o + `dragonkue/bge-reranker-v2-m3-ko`)	Colab T4 + OpenAI API	Notebook ready; needs an API key
LF-Eval (G-Eval with an LLM judge)	needs an LLM API	Not run

Data: 226 questions (57 one-hop, 127 two-hop, 42 three-hop) and 233,544 Korean statute paragraphs, both in `experiments/source/`. Every question also has an English translation, used for the examples below.

Open question: which API budget and model should the full run use? GPT-4o matches the paper; a cheaper model would no longer be a like-for-like reproduction.

Results

The paper's +37.91 matches the retrieval F1 gap in the saved outputs (59.41 vs 21.50, a gap of 37.90); on answer F1, ParSeR leads standard retrieval by 19.4 and no-retrieval prompting by 9.9.



results/*_gpt4o.jsonl in the repo, scored with lf-eval/eval_pipeline.py functions · 226 questions

Standard retrieval (SP + BM25) scores below answering with no retrieval at all: wrong provisions mislead GPT-4o more than no provisions. Self-Ask's 7.8 comes from answers cut down to fragments such as "1년" ("1 year"), likely an answer-extraction problem rather than the method itself.

Re-running the baseline from scratch. BM25 over 233,544 provisions, query = background + question, k = gold hop count, as in `naive/openai/sp_or_openai.py`:

Run	Precision	Recall	F1	Exact match
Authors' saved retrieval	—	21.0	21.5	—
My re-run	22.4	20.9	21.4	7.1

The retrieved provisions are identical for 218 of 226 questions; the 8 differences are most likely ties in BM25 scores broken differently across library versions.

By number of hops (GPT-4o, saved outputs). ParSeR's lead is largest on 1-hop questions and narrows as more provisions are needed:

Hops	Questions	ParSeR answer F1	ParSeR retrieval F1	Standard answer F1	Standard retrieval F1
1	57	43.6	77.8	12.0	17.5
2	127	50.5	55.4	33.2	25.5
3	42	36.3	46.7	27.2	14.8

The full table for all 46 saved runs (3 backbones × methods) is in `scores.txt`.

Issues found in the repository

The most serious is a selection bug in the cleaned-up ParSeR code; the rest stop the scripts from running as documented.

#	Where	Problem	Effect	Fix used
1	`parser/openai/selection_retrieval.py` line 71, `parser/vllm/selection_retrieval.py` line 117	`contexts_list[0][choice]` indexes the first sub-query's candidates for every sub-query; the original `2.selection_retrieval_gpt.py` uses `re[i]`	Multi-hop questions get provisions picked from the wrong candidate list	Use the i-th list; the notebook scores both from the same LLM outputs
2	`lf-eval/eval_pipeline.py`	Reads `item['answer']`; saved results store `answers` (a list of dicts)	`KeyError: 'answer'` on every file in `results/`	Own scorer that imports the same `compute_token_f1` and `evaluate_retrieval`

#	Where	Problem	Effect	Fix used
3	`parser/openai/run_pipeline.py`	Paths `source/koblex.jsonl` and `source/statute.jsonl`, but data is in `experiments/source/`	Pipeline fails at step 1 when run from its folder	Absolute paths
4	`parser/openai/utils.py`	`aiohttp.ClientTimeout(10)` is a 10 s total limit; after 20 retries a call returns `None`, which the parser unpacks	Long GPT-4o calls time out, then a crash	120 s timeout, failures become empty strings
5	`parser/openai/utils.py` `get_reranker`	`default_activation_function=` is rejected by sentence-transformers 6.x (now `activation_fn=`)	`TypeError` on a current Colab	Try the new name, fall back to the old
6	`parser/openai/question_answering.py`	`'\n'.join(...)` inside an f-string expression	`SyntaxError` on Python 3.11 or older (fine on 3.12+)	Build the string outside the f-string
7	`naive/openai/sp_or_openai.py`	BM25 retrieves `k = n_hops` taken from the gold label	The baseline is told how many provisions the answer needs, which a real system would not know	Kept as is, to match the paper; flagged
8	`experiments/README.md`	Commands use `self-ask/` (folder is `self_ask/`) and run `.sh` files with `python`	Copy-pasted commands fail	Corrected commands

Also noted: `parser/original/2.selection_retrieval_gpt.py` builds a separate BM25 index per law but then always searches the global index (`retr = global_retriever`), so the per-law indexes are unused. `statute_eng.jsonl`, listed in the README, is on Hugging Face only, not in the repo.

Failure analysis

Standard retrieval fails by fetching the wrong law entirely; ParSeR fails by fetching the right law's neighbouring provision, which produces confident, specific, wrong answers. Examples below use the saved GPT-4o outputs and the benchmark's English translations.

Where standard retrieval fails and ParSeR succeeds

Question (English)	Gold provision	Standard retrieval fetched	Standard answer
What punishment could A face for secretly entering B's house at night and stealing a gold necklace? (`qa_92_1hop_149`)	Criminal Act art. 330, night-time burglary	Road Traffic Act art. 26, yielding at uncontrolled junctions	Says the context has nothing on theft
Can A vote on election day? (`qa_139_1hop_254`)	Public Official Election Act art. 18, persons without voting rights	Same Act, art. 6, guarantee of voting rights	Says the context does not cover prisoners
What legal effect do court-recorded settlement minutes have? (`qa_172_1hop_308`)	Civil Procedure Act art. 220	Arbitration Act art. 31	Wrongly applies the arbitration rule

In all three, ParSeR retrieved exactly the gold provision. Its token F1 was still only 0.24 to 0.36, which shows the metric's weakness: a correct answer worded differently from the reference scores low.

Where ParSeR fails

Question (English)	Gold provision	ParSeR fetched	Result
Until when can B claim damages from the carrier A? (`qa_19_1hop_28`)	Commercial Act art. 814: claims lapse after **1 year**	Commercial Act art. 919: carrier's claims lapse after **2 years**	Answers "2 years": a fluent, specific hallucination grounded in a real but wrong provision
Does A have standing to sue, and when is a stay granted? (`qa_621_2hop_937`)	Administrative Litigation Act arts. 12 and 23(2)	Same Act, arts. 18(2) and 23(1)	Right law, wrong paragraphs for both hops

Question (English)	Gold provision	ParSeR fetched	Result
Who gets the reward, and what is the annual cap? (`qa_163_2hop_630_rand`)	Framework Act on National Taxes Enforcement Decree art. 65-4, paras. 6 and 21	The parent Act art. 84-2 and the Decree art. 65-4 para. 18	Right article family, wrong paragraphs; states a 50 million KRW annual cap where the gold answer says 1 million KRW

ParSeR retrieved none of the gold provisions for 36 of 226 questions. No ParSeR answer refused, while 26 standard-retrieval answers did. ParSeR is less likely to say "I don't know" and more likely to be wrong with confidence when retrieval misses, which matters for a hallucination-resistant system.

Own experiment: Korean-aware BM25

Tokenizing Korean into character bigrams nearly doubles BM25's retrieval F1 (21.4 to 37.4) with no LLM and no GPU.

Why try it: the repo calls `bm25s.tokenize(..., stopwords="en")` and an English stemmer. Korean attaches particles to words (`손해배상청구권` + `을`), so the query word `손해배상청구권을` never matches `손해배상청구권은` in a statute. Bigrams (`손해` , `해배` , `배상` …) match the shared stem.

Setup: the same 233,544 provisions, query = background + question, as in the standard-retrieval baseline.

BM25 tokenizer	F1 (k = gold hops)	Recall@10	Recall@30	Recall@100
Repo: English tokenizer + stemmer	21.4	32.2	41.6	51.8
Korean character bigrams	37.4	53.2	66.2	75.6

Two consequences:

- **A fairer baseline.** Part of ParSeR's +37.9 lead over standard retrieval comes from a weak tokenizer, not from the method. With bigrams, the gap to ParSeR's 59.4 shrinks to 22.0.

- **Likely a free gain for ParSeR too.** ParSeR's stage 2 uses the same English tokenizer to pull its top 100 candidates, so anything missed there can never be reranked or selected. Section 8 of the notebook re-runs ParSeR with bigram BM25; only the selection calls cost extra.

Caveat: bigrams are a crude stand-in for a Korean morphological analyser (e.g. Kiwi or Mecab-ko), which should do better still.

What this means for NyayaMarg, and next steps

The ParSeR idea (let the LLM draft the law it expects, then search with that draft) should carry over to Indian statutes, but tokenization and evaluation need rethinking first.

- **Corpus.** KoBLEX works at paragraph (항) level over one statute book. An Indian equivalent needs central Acts split to section and sub-section level, ideally including the new BNS, BNSS and BSA alongside the IPC, CrPC and Evidence Act they replaced, since questions may cite either.
- **Language.** English statutes avoid the Korean tokenizer problem, but Hindi and other Indian-language versions would hit the same issue (agglutination, script mixing). Test the tokenizer before anything else.
- **Near-miss hallucination.** The art. 814 vs 919 case shows the risk: a real, plausible, wrong provision. A citation check that compares the generated answer's specific facts (periods, amounts, sections) against the retrieved text would target this directly.
- **Evaluation.** Token F1 punishes correct answers that are worded differently. Retrieval F1 plus an LLM judge (as in LF-Eval), spot-checked by a human, gives a truer picture.
- **Baselines.** Use a properly tokenized BM25 and a dense retriever, and do not give the baseline the gold number of hops.

Next steps:

- [ ] Run the notebook on 20 questions with GPT-4o; check cost and compare with the saved ParSeR numbers
- [ ] If within budget, run all 226 and add the fixed vs buggy selection comparison
- [ ] Run section 8 (ParSeR + bigram BM25)
- [ ] Ask ma'am which Indian legal corpus and benchmark NyayaMarg is using

Meeting prep: likely questions

Why generate a fake provision before searching? A question is written in everyday words ("until when can B sue"); statutes use legal phrasing ("claims lapse if not exercised within"). The LLM's draft provision is written in statute language, so it matches real provisions better than the question does. Retrieval F1 59.4 vs 21.5 shows it works.

Why does the draft not cause hallucination itself? It is only a search query. The answer is generated from the real provisions that were retrieved, never from the draft.

Why three stages and not just top-1 from the reranker? BM25 is cheap but rough, so it narrows 233,544 provisions to 100. The cross-encoder reads query and provision together and reorders those 100. The LLM then picks one of the top 10, because it can use the whole scenario, which the reranker never sees (it scores only the question).

What is multi-hop here? An answer needing 2 or 3 provisions, often from different articles. ParSeR handles it by generating one draft per provision needed and retrieving for each separately.

What exactly did you reproduce? The scores of all saved outputs and the BM25 baseline from scratch (21.4 vs 21.5, 218 of 226 identical). The full LLM run is prepared but needs API access.

What would you change? Fix the selection bug, use Korean-aware tokenization in BM25 (+16 F1 on the baseline alone), and add a check that the answer's specific facts appear in the retrieved text.

Weak points to admit. Token F1 is a poor measure for long legal answers; LF-Eval was not re-run; results use one model per run and no repeated trials.